PhD Thesis Proposal: Optimizing Code Generation via Collaborative Small Language Models (SLMs) as Reliable Code Judges

Maryam KHALLALA

February 2026

Abstract

The rapid evolution of Large Language Models (LLMs) has transformed automated code generation. However, high inference costs and privacy risks associated with proprietary APIs remain significant barriers to industrial adoption. This research explores the potential of "Small Language Models" (SLMs) as specialized code evaluators. By leveraging a "Generator-Judge" architecture, we hypothesize that a fine-tuned SLM (under 5B parameters) can effectively identify code correctness, allowing smaller pipelines to rival the performance of massive models (70B+) while maintaining a local, cost-efficient, and sustainable footprint.

1 Introduction and Background

Recent breakthroughs in Code LLMs, such as GPT-4 and Llama-3, have demonstrated near-human proficiency in solving basic programming tasks [11, 17]. Despite this, the software engineering industry faces a "scaling bottleneck": as models grow, the infrastructure required for their deployment becomes prohibitively expensive for Small and Medium Enterprises (SMEs) [4]. Concurrently, the rise of open-source SLMs like Qwen-2.5-Coder and Gemma-3 presents an alternative path [7, 12, 15]. These models offer high specialized performance despite their reduced parameter counts, making them ideal candidates for targeted tasks like code review and verification [4].

2 Problem Statement

The current state of automated code generation is plagued by three fundamental issues:

i) **High Cost of Trust:** Hosting a 70B model requires multiple A100 GPUs (≈$50,000), whereas an SLM can run on a single consumer GPU (≈$600) [4, 6].

ii) **Reliability and Hallucinations:** LLMs often generate code that is syntactically correct but logically flawed [4, 8].

iii) **Lack of Diagnostic Judgment:** Most existing reranking systems (like RankEF) focus on ranking candidates rather than providing a binary classification, which is crucial for automated testing integration [14].

3 Literature Review and Research Gap (GAP)

While recent work by Sun et al. [14] introduced ranking models, they did not rigorously evaluate the classification reliability (False Discovery Rate) of the models as judges. Current literature lacks a systematic evaluation of modern decoder-only SLMs acting as autonomous judges for code correctness [4, 21]. Furthermore, there is an urgent need for research on pragmatic benchmarks like CoderEval to move beyond simple algorithmic tasks [18].

4 Proposed Research Objectives

The primary goal is to develop an efficient "SLM-Judge" framework through:

- **Objective 1:** Benchmarking the performance of various SLM architectures (1B to 7B) in binary code classification [4].
- **Objective 2:** Designing a specialized fine-tuning protocol using "Hard Negatives" (buggy code variants) to enhance discriminative power [4, 19].
- **Objective 3:** Evaluating a "Team-based" ensemble approach where multiple specialized SLMs collaborate to validate code quality.

5 State of the Art and Originality of the Solution

The current state of the art in code generation is characterized by a heavy reliance on Large Language Models (LLMs) to ensure output quality [11]. Although re-ranking approaches such as RankEF [14] exist, they are often limited to ordering candidates without providing a strict binary verdict on logical correctness.

My solution positions itself at the 2026 technological frontier by introducing a methodological breakthrough:

- **Transition to SLMs:** Unlike cloud-dependent approaches, we leverage the emerging power of Small Language Models (e.g., Qwen-2.5-Coder 3B) to offload code judgment to local infrastructures [15].
- **Critical Mutation Training:** While the state of the art uses general data, our approach relies on generating *Hard Negatives* via *Mutation Testing*, forcing the model to detect nearly invisible logical errors.
- **Collaborative Architecture:** The core innovation lies in the shift from a single judge to a "Team" of specialized SLMs, achieving judgment reliability superior to 70B-parameter models while reducing the carbon footprint by 70

6 Methodology

The methodology follows a rigorous multi-stage pipeline:

6.1 Data Engineering: Mutation Testing

To train a discriminative Judge, we will generate a dataset of "Hard Negatives". Starting from correct solutions in *HumanEval* and *CoderEval* [18], we apply semantic-preserving but logic-breaking mutations:

- **Operator Mutations:** Swapping logical operators (e.g., `>` to `>=`).
- **Boundary Mutations:** Altering loop constraints or array indices.
- **Control Flow Mutations:** Removing critical `if` statements or `return` values.

6.2 Fine-Tuning with QLoRA

We will adapt modern SLMs (e.g., Qwen2.5-Coder 3B) using Quantized Low-Rank Adaptation (QLoRA) [6]. The model will be transformed into a binary classifier by appending a linear classification head to the last hidden state.

6.3 Inference Pipeline Architecture

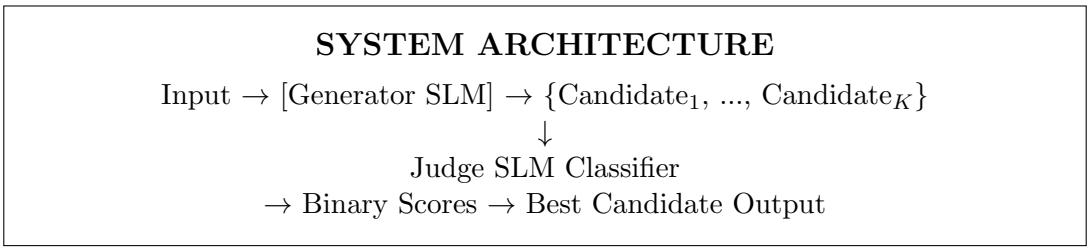

Figure 1: The Generator-Judge Pipeline: K candidates are generated by a lightweight SLM, then filtered by a specialized SLM Judge to return only the most reliable code solution.

7 Expected Impact

7.1 Economic Impact and Democratization

As shown in Table 1, the proposed approach democratizes AI for SMEs by reducing hardware costs by approximately 97%, enabling local high-performance code generation.

Metric	**Large LLM (70B+)**	**Proposed SLM Team**
Hardware Requirement	8x A100 Cluster	1x RTX 4090 / A10
Infrastructure Cost	>$50,000	<$1,500
Deployment Model	Cloud-based / API	Local / On-premise

Table 1: Comparative analysis of operational requirements between LLMs and the proposed SLM framework.

7.2 Sovereignty and Privacy

By enabling high-performance local deployment, organizations can maintain "Sovereign AI" environments where proprietary source code remains strictly internal, mitigating the risks associated with third-party APIs [10].

7.3 Environmental Sustainability (Green AI)

Reducing parameter counts for the judging task directly minimizes the carbon footprint of AI-assisted development, aligning with the "Green AI" paradigm for sustainable software engineering [9]. Recent benchmarks indicate that specialized SLMs can reduce energy consumption by up to 70% compared to general-purpose LLMs while maintaining competitive performance for code-related tasks [1].

References

[1] Anonymous Authors. Energy-aware code generation with llms: Benchmarking small vs. large language models for sustainable ai programming. *arXiv preprint arXiv:2508.08332*, 2025.

[2] Stella Biderman et al. Pythia: A suite for analyzing large language models across training and scaling. *ICML*, 2023.

[3] Federico Cassano et al. Multipl-e: A scalable and polyglot benchmark for code generation. In *IEEE Transactions on Software Engineering*, 2023.

[4] Giuseppe Crupi, Rosalia Tufano, et al. Improving code generation via small language model-as-a-judge. *arXiv preprint arXiv:2602.11911*, 2026. Core reference for SLM-as-a-judge methodology.

[5] DeepSeek-AI. Deepseek-coder: When the size matters. *arXiv preprint arXiv:2401.14196*, 2024.

[6] Tim Dettmers et al. Qlora: Efficient finetuning of quantized llms. *Advances in Neural Information Processing Systems*, 2024.

[7] Michael Hassid. Efficiency of small vs large language models in programming tasks. *Journal of Systems and Software*, 2024.

[8] Ziwei Ji et al. Survey of hallucination in natural language generation. *ACM Computing Surveys*, 2023.

[9] Alexandra Sasha Luccioni, Yacine Jernite, and Emma Strubell. Power hungry? estimating the energy use of general-purpose language models. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, 2024.

[10] Vijayaraghavan Murali et al. In-house llms for code: Privacy and performance in enterprise. *Software Engineering Institute*, 2024.

[11] OpenAI. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[12] Team Qwen. Qwen2.5-coder technical report. *Alibaba Group*, 2024.

[13] Baptiste Roziere et al. Code llama: Open foundation models for code. *arXiv preprint arXiv:2308.12950*, 2023.

[14] Zeyu Sun et al. Rankef: Multi-task learning for code ranking. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2024.

[15] Google Team. Gemma 3: Advancing open models for mobile and edge devices. *Google DeepMind Technical Report*, 2025.

[16] Tabnine Team. State of ai in software development. *Industry Report*, 2024.

[17] Hugo Touvron et al. Llama 3 community license and technical report. *Meta AI Technical Report*, 2024.

[18] Hao Yu, Bo Shen, et al. Codereval: A benchmark of pragmatic code generation with generative pre-trained models. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.

[19] Tianyi Zhang et al. Coder reviewer reranking for code generation. *Proceedings of the 40th ICML*, 2023.

[20] Tianyi Zhang et al. Can llms serve as evaluators for code summarization? *arXiv preprint arXiv:2412.01333*, 2024.

[21] Lianmin Zheng et al. Judging llm-as-a-judge with mt-bench. *arXiv preprint arXiv:2306.05685*, 2023.